# System Architecture Document

## Version 1 Real-Time YOLO Object Detection Application

Document Status: Architecture Specification
Version: 1.0
Application Type: Local Python Computer-Vision Application
Primary Function: Real-time object detection from a live camera feed

---

## 1. Purpose

This document defines the System Architecture for Version 1 of a local Python application that captures video from a camera, performs real-time YOLO object detection, and displays the resulting detections over the live video feed.

The architecture is intentionally modular. V1 provides a complete end-to-end product while establishing boundaries that allow later replacement or extension of individual subsystems without requiring a fundamental redesign.

The system does not require networking, cloud services, databases, remote APIs, or persistent image storage.

This document specifies architecture and behavioral requirements. It does not specify implementation-level source code.

---

# 2. Product Objective

The V1 system shall:

1. Initialize the application.
2. Select and open a local camera.
3. Continuously capture frames.
4. Validate captured frames.
5. Perform any required lightweight preprocessing.
6. Pass valid frames to a pretrained YOLO model.
7. Obtain object detections from the model.
8. Extract:
    - Class ID
    - Class name

    - Confidence
    - Bounding-box coordinates
9. Validate and normalize detection results.
10. Render bounding boxes and labels onto the live frame.
11. Display the resulting frame.
12. Accept a user shutdown request.
13. Recover from defined transient failures where practical.
14. Escalate unrecoverable failures.
15. Release all acquired resources during shutdown.
16. Record sufficient diagnostic information through centralized logging.

No custom model training or model-management platform is required.

---

# 3. Architectural Goals

The architecture prioritizes:

- Correctness
- Fault tolerance
- Maintainability
- Testability
- Clear separation of concerns
- Predictable lifecycle behavior
- Controlled resource usage
- Useful observability
- Local/offline operation
- Straightforward future extension

The architecture deliberately avoids:

- Microservices
- Distributed processing
- Databases
- Networking layers
- Message brokers
- Web frameworks

- Unnecessary dependency injection frameworks
- Excessive threading
- Persistent frame pipelines
- Complex orchestration frameworks

V1 is a local application and should remain proportionate to that problem.

---

# 4. Architectural Assumptions

The following assumptions are made where the requirements do not specify an exact implementation:

### A-001: YOLO implementation

Ultralytics YOLO is the preferred V1 implementation because it provides a practical Python interface for loading pretrained YOLO models and performing inference.

The architecture does not make the rest of the application dependent on Ultralytics-specific result objects. Model-specific output is converted into an application-owned detection representation at the detector boundary.

This permits a future detector implementation to use another YOLO implementation or inference backend.

### A-002: Camera interface

V1 uses OpenCV's camera interface, normally through `cv2.VideoCapture`.

The application defines its own `Camera` abstraction around OpenCV rather than exposing `cv2.VideoCapture` throughout the application.

### A-003: Processing architecture

V1 uses a single primary processing loop unless performance testing demonstrates that this is insufficient.

Camera capture, preprocessing, inference, detection processing, rendering, and display occur sequentially for each processed frame.

### A-004: GPU

The application supports CPU and GPU execution where supported by the selected YOLO/PyTorch environment.

Device selection is configuration-driven.

V1 does not require automatic GPU-to-CPU failover unless explicitly enabled by configuration. Silent fallback can conceal deployment problems and make performance unpredictable.

### A-005: Frame persistence

Captured frames are transient runtime data. V1 does not persist camera frames or detection images.

### A-006: Configuration

V1 uses centralized Python configuration rather than introducing a configuration framework.

A future release may add YAML, TOML, JSON, environment-variable configuration, or a GUI without changing the internal component boundaries.

### A-007: Performance target

Exact real-time performance depends substantially on:

- Camera resolution
- YOLO model size
- CPU/GPU
- Driver/runtime configuration
- Input frame rate
- Number of detections

Therefore, architecture-level targets are specified where useful, while model/hardware-specific performance limits must be established through benchmarking.

---

# 5. Technology Architecture

## 5.1 Python

Python is the application programming language.

Responsibilities:

- Application orchestration
- Configuration
- Error handling
- Lifecycle management
- Module integration
- Logging
- Runtime state

Python standard-library facilities should be preferred where they adequately solve the problem.

Relevant standard-library modules include:

- `logging`
- `time`
- `pathlib`
- `enum`
- `dataclasses`
- `typing`
- `traceback`
- `sys`
- `os`
- `contextlib`

---

## 5.2 OpenCV

OpenCV provides the local computer-vision and camera/display layer.

Primary responsibilities:

- Camera access
- Frame capture
- Basic image operations where necessary
- Window creation
- Frame display
- Keyboard input
- Bounding-box rendering
- Text rendering

OpenCV is not responsible for:

- Application lifecycle
- YOLO model management
- Application configuration
- Centralized logging
- Detection business logic

---

## 5.3 Ultralytics YOLO

Ultralytics YOLO is the preferred V1 model interface.

Responsibilities:

- Loading a pretrained YOLO model
- Running inference
- Providing model-specific detection output

The detector adapter is responsible for converting the library's native output into application-owned detection objects.

The remainder of the application must not depend directly on Ultralytics result structures.

---

## 5.4 PyTorch / Model Runtime

PyTorch or the runtime required by the selected YOLO implementation provides the deep-learning execution environment.

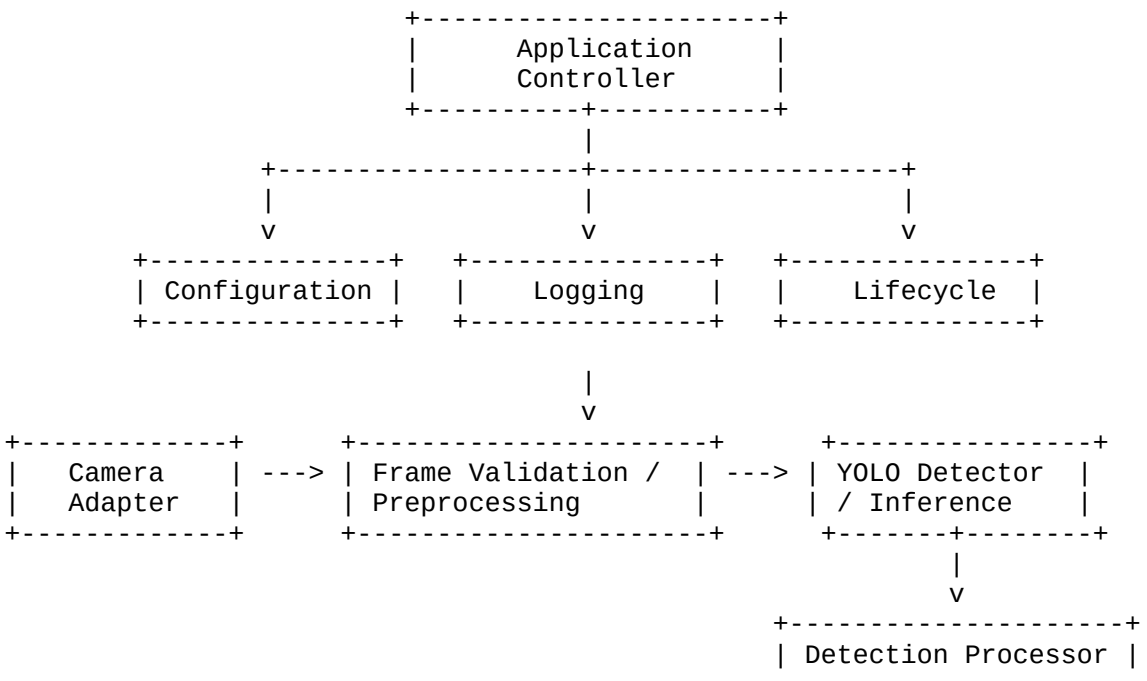
Responsibilities include:

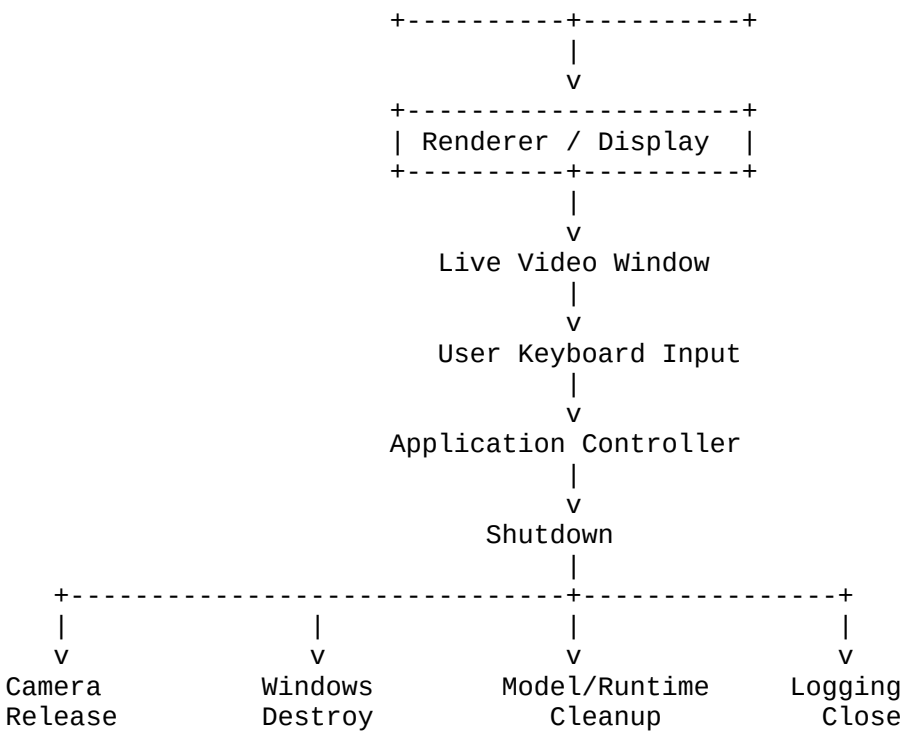
- Tensor computation
- CPU execution
- CUDA/GPU execution where available
- Model execution

The application should treat the deep-learning runtime as an infrastructure dependency rather than allowing runtime-specific details to spread throughout the architecture.

---

# 6. High-Level Architecture






The controller coordinates components but does not perform their specialized work.

---

# 7. Component Architecture

## 7.1 Application Controller

Suggested module:

`main.py` or `application.py`

Responsibility:

- Own application lifecycle.
- Coordinate startup.
- Construct required components.
- Transition application state.
- Execute the primary processing loop.
- Coordinate error recovery.
- Detect shutdown conditions.
- Initiate cleanup.
- Provide top-level exception protection.

The controller must not:

- Contain YOLO-specific parsing logic.

- Contain camera implementation details.
- Draw bounding boxes directly.
- Implement logging internals.
- Contain large amounts of image-processing logic.

### Conceptual interface

```
Application.start()
Application.run()
Application.request_shutdown(reason)
Application.shutdown()
```

The exact implementation may use functions rather than a class if that produces a simpler design.

---

# 8. Configuration Component

Suggested module:

`config.py`

Responsibility:

- Store validated operational configuration.
- Provide centralized application settings.
- Prevent operational parameters from being scattered throughout the codebase.

Configuration should include:

```
camera_index
camera_width
camera_height
camera_fps (if supported)
model_path
model_name
device
confidence_threshold
exit_key
window_name

camera_max_read_failures
camera_retry_delay
camera_reconnect_attempts
camera_reconnect_delay

log_level
log_directory
log_file
log_rotation_policy

performance_logging_enabled
performance_log_interval
debug_detection_logging
```

Configuration should be loaded and validated before normal execution.

Invalid configuration is a startup failure.

Configuration should not contain transient runtime state.

For example:

```
Configuration:
    confidence_threshold = 0.50

Runtime state:
    current_camera_state = OPEN
```

These must remain conceptually separate.

---

# 9. Camera Component

Suggested module:

`camera.py`

The camera subsystem provides the sole application-facing interface to the camera.

## Responsibilities

- Select camera.
- Open camera.
- Configure supported properties.
- Determine whether the camera is open.
- Capture frames.
- Validate basic camera read success.
- Track camera state.
- Report read failures.
- Perform bounded recovery.
- Release the camera.

## Conceptual interface

```
Camera.open()
Camera.is_open()
Camera.read()
Camera.reconnect()
Camera.release()
Camera.state
```

`read()` should return a structured result rather than forcing the controller to interpret OpenCV-specific behavior everywhere.

Conceptually:

```
FrameResult
    success
    frame
    error/reason
```

The exact class is an implementation decision.

---

# 10. Camera State

The camera should maintain a small explicit state model.

Possible states:

```
CLOSED
OPENING
OPEN
READ_FAILURE
RECOVERING
FAILED
RELEASING
```

Normal transition:

```
CLOSED
  |
  v
OPENING
  |
  +---- failure ---> FAILED
  |
  v
OPEN
  |
  +---- read failure ---> READ_FAILURE
                               |
                               v
                          RECOVERING
                           /       \
                      success     failure limit
                        |             |
                        v             v
                       OPEN         FAILED
```

---

# 11. Camera Failure Handling

## Camera cannot be opened

Detection:

- `VideoCapture` reports failure.
- Camera does not reach the open state.
- Initialization raises an exception.

Behavior:

1. Log the failure.
2. Do not enter the normal frame-processing loop.
3. Release any partially initialized camera resources.
4. Transition to failed/shutdown state.
5. Return a meaningful failure status.

This is normally unrecoverable during V1 startup unless explicit bounded camera discovery/retry is configured.

---

## Camera already in use

This is treated as an operational camera initialization failure.

The application should:

- Log that the camera could not be opened.
- Avoid infinite attempts.
- Terminate startup cleanly.

The system should not attempt to forcibly terminate another application's camera usage.

---

## Single read failure

A single `read()` failure is potentially transient.

Behavior:

1. Increment consecutive read-failure count.
2. Log a warning.
3. Do not replace the previous valid frame with invalid data.
4. Apply the configured retry/recovery delay.
5. Continue if the camera subsequently recovers.

A single failure should not immediately terminate the application.

---

## Repeated read failures

A configurable consecutive failure limit is required.

Example architectural behavior:

```
read failure
     |
     v
increment consecutive failures
     |
     +---- below limit ----> retry/recover
     |
     +---- reaches limit --> recovery attempt
                                |
                                +-- success --> reset counter
                                |
                                +-- failure --> application shutdown
```

The exact retry count must be configurable and established through testing.

A reasonable V1 starting point is a small bounded number of consecutive failures rather than an indefinite retry loop.

---

## Camera disconnect

A disconnect is detected through repeated unsuccessful reads or camera API failure.

Recovery:

1. Mark camera as unavailable.
2. Log the beginning of the failure.
3. Stop treating frames as valid.
4. Attempt bounded reconnection.
5. Wait between attempts.
6. Reinitialize camera state if reconnection succeeds.
7. Reset failure counters only after confirmed successful operation.
8. Resume processing.
9. If recovery fails after the configured limit, shut down.

A single successful `read()` should not necessarily reset all recovery state. Recovery should be considered successful after a valid camera state has been re-established.

---

# 12. Frame Validation and Preprocessing

Suggested module:

`preprocessing.py`

Responsibility:

- Validate frame existence.
- Validate frame dimensions.
- Validate frame shape.
- Validate frame data type.
- Perform only preprocessing required by the model pipeline.
- Reject malformed frames safely.

Validation should include at minimum:

```
frame is not None
frame is not empty
frame has expected dimensionality
frame has valid image shape
frame contains valid data type
```

Invalid frames are runtime input failures, not necessarily application failures.

Behavior:

```
Invalid frame
    |
    v
Log warning/debug information
    |
    v
Skip frame
    |
    v
Do NOT update state using invalid data
    |
    v
Capture next frame
```

Invalid frame handling must not create an infinite loop independent of camera state. If invalid frames continue indefinitely, the camera subsystem should eventually classify the condition as an unrecoverable camera/input failure.

---

# 13. YOLO Detector Component

Suggested module:

`detector.py`

The detector is responsible for model lifecycle and inference.

## Responsibilities

- Load the configured model.

- Validate model initialization.
- Configure inference device.
- Execute inference.
- Convert model output into application-level detections.
- Handle model-specific exceptions.
- Track inference performance.
- Release model-related resources when applicable.

## Conceptual interface

```
Detector.load()
Detector.is_ready()
Detector.detect(frame)
Detector.release()
```

The detector should expose application-level results rather than raw library objects.

---

# 14. Detection Data Model

The application should define an internal detection representation.

Conceptually:

```
Detection
    class_id
    class_name
    confidence
    x1
    y1
    x2
    y2
```

Optional future-compatible fields may include:

```
model_id
inference_timestamp
detection_id
```

These should not be populated merely for future speculation in V1.

A V1 detection therefore minimally represents:

```
Class
Confidence
Bounding box
```

The bounding box coordinate convention must be defined consistently throughout the application.

Recommended representation:

```
x1, y1 = top-left
```

```
x2, y2 = bottom-right
```

Coordinates should use the coordinate system of the displayed/processed frame.

---

# 15. Model Loading Failure Handling

Possible causes:

- Missing model file.
- Invalid model path.
- Corrupted model.
- Unsupported model format.
- Missing dependency.
- Runtime incompatibility.
- Unsupported device.
- GPU initialization failure.
- Insufficient system resources.

Model initialization occurs before entering the normal runtime loop.

If the model cannot be loaded:

```
Model initialization failure
        |
        v
Log detailed exception
        |
        v
Do not enter inference loop
        |
        v
Release initialized resources
        |
        v
Shutdown
```

A model that cannot initialize is a startup-critical failure.

The application should not repeatedly reload a known-invalid model indefinitely.

---

# 16. GPU Handling

Device selection is explicit.

Possible configuration:

```
cpu
```

```
cuda
auto
```

If V1 supports `auto`, the behavior must be deterministic and logged.

For explicit `cuda` selection:

```
CUDA requested
     |
     +-- available --> initialize GPU runtime
     |
     +-- unavailable --> initialization failure
```

Silent fallback from CUDA to CPU is not recommended for explicit GPU configuration because it can conceal deployment and performance problems.

If `auto` is supported:

```
Auto device selection
       |
       +--> supported GPU available --> GPU
       |
       +--> otherwise ---------------> CPU
```

The selected device must be logged during startup.

---

# 17. Per-Frame Inference Errors

A model inference exception is different from a model initialization failure.

For a single inference error:

1. Catch the exception at the detector boundary.
2. Record diagnostic information.
3. Mark that frame's inference as failed.
4. Do not expose raw model exceptions to unrelated components.
5. Determine whether the detector remains usable.
6. Continue if the error is classified as transient and the detector remains valid.
7. Escalate if the model/runtime has become unusable.

A malformed frame should ideally be rejected before inference.

Repeated inference failures should not result in:

```
try inference
catch
try inference
catch
try inference
catch
...
```

without a termination condition.

A configurable consecutive inference-failure threshold should be used if recovery is permitted.

---

# 18. Detection Processing Component

Suggested module:

`detection.py` or `detection_processor.py`

Responsibility:

- Convert detector results into validated application-level detections.
- Apply confidence filtering.
- Validate bounding boxes.
- Clamp or reject invalid coordinates according to defined policy.
- Handle empty detection sets.
- Preserve multiple detections.

The component must not perform camera management or rendering.

---

# 19. Confidence Threshold

A configurable confidence threshold is applied to model detections.

Example conceptual rule:

```
confidence < configured threshold
        |
        v
discard detection

confidence >= threshold
        |
        v
retain detection
```

The threshold should be applied consistently.

The system must distinguish:

```
Model returned zero detections
```

from:

```
Model returned detections, but all were filtered
```

Both are normal runtime conditions.

Neither should be logged as an error.

---

# 20. Bounding-Box Validation

Each detection should be validated before rendering.

Validation should check:

- Coordinates are numeric.
- Coordinates are finite.
- `x1 <= x2`.
- `y1 <= y2`.
- Bounding box has non-zero or meaningful area.
- Coordinates are compatible with the frame dimensions.

For coordinates slightly outside frame boundaries, the architecture may clamp them:

```
x1 = max(0, x1)
y1 = max(0, y1)
x2 = min(frame_width, x2)
y2 = min(frame_height, y2)
```

For fundamentally malformed boxes, the detection should be rejected.

Invalid individual detections should not necessarily invalidate the entire frame.

---

# 21. Multiple Detections

The detector returns zero or more detections per frame.

Examples:

```
Frame A:
    person
    person
    bicycle

Frame B:
    no detections

Frame C:
    car
    car
    truck
    person
```

The architecture must not assume one object per frame.

Each valid detection is independently validated and passed to rendering.

There is no object identity tracking in V1.

Two `person` detections in different frames are simply detections. The application must not attempt to determine whether they represent the same physical object.

---

# 22. Large Numbers of Detections

The application should process all valid model detections returned for a frame, subject to configured model behavior and reasonable resource constraints.

V1 does not implement sophisticated detection prioritization.

If extreme detection counts produce unacceptable performance, this is a performance/configuration concern rather than justification for silently discarding detections.

Any future detection limiting policy should be explicit and configurable.

---

# 23. Renderer and Display Component

Suggested module:

`renderer.py`

Responsibilities:

- Draw bounding boxes.
- Draw class labels.
- Draw confidence values.
- Produce the display frame.
- Display the frame.
- Process user keyboard input associated with the display.

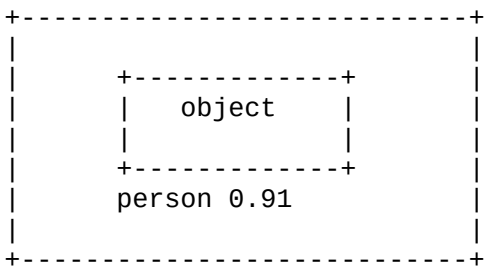
The renderer must not:

- Open the camera.
- Run YOLO inference.
- Modify detector state.
- Perform model loading.
- Manage application shutdown directly.

---

# 24. Detection Visualization

For each valid detection:




The label should contain:

```
<class name> <confidence>
```

Confidence formatting should be consistent, for example:

```
person 0.91
car 0.87
dog 0.73
```

The exact visual style is not architecturally significant.

---

# 25. Display Failure

OpenCV display operations can fail due to:

- Unsupported display environment.
- Window initialization failure.
- Desktop/session problems.
- OpenCV runtime errors.
- Window closure.

A display failure during runtime should be treated as an application-level failure unless a safe alternative display behavior exists.

V1 has no alternate UI, so an unrecoverable display failure normally results in shutdown.

The failure must still pass through centralized error handling and cleanup.

---

# 26. User Input

The display layer detects the configured exit key.

Recommended default:

```
Q
```

The exact key is configurable.

Conceptual control flow:

```
Display frame
      |
      v
Read keyboard state
      |
      +---- exit key --> request shutdown
      |
      +---- other key --> continue
```

User input must not be the only mechanism for cleanup. Unexpected exceptions and system interruption must also result in controlled shutdown.

---

# 27. Application State Model

The application should use an explicit lifecycle state model.

Recommended states:

```
STARTING
INITIALIZING_CAMERA
INITIALIZING_MODEL
RUNNING
RECOVERING
SHUTTING_DOWN
STOPPED
FAILED
```

A separate component may maintain subsystem states such as:

```
Camera:
    CLOSED
    OPEN
    RECOVERING
    FAILED

Detector:
    UNINITIALIZED
    READY
    FAILED

Renderer:
    UNINITIALIZED
    READY
    FAILED
```

The application state should represent overall lifecycle rather than duplicating every subsystem state.

---

# 28. State Transitions

Normal startup:

```
STARTING
   |
```

```
   v
INITIALIZING_CAMERA
   |
   v
INITIALIZING_MODEL
   |
   v
RUNNING
```

Normal shutdown:

```
RUNNING
   |
   v
SHUTTING_DOWN
   |
   v
STOPPED
```

Recoverable failure:

```
RUNNING
   |
   v
RECOVERING
   |
   +---- recovered ----> RUNNING
   |
   +---- unrecoverable -> SHUTTING_DOWN
```

Fatal failure:

```
RUNNING
   |
   v
FAILED
   |
   v
SHUTTING_DOWN
   |
   v
STOPPED
```

Unexpected exception:

```
ANY ACTIVE STATE
       |
       v
Top-level exception handler
       |
       v
SHUTTING_DOWN
       |
       v
STOPPED / FAILED
```

Invalid state transitions should be prevented by the application controller.

---

# 29. Main Runtime Data Flow

The primary data flow is:

```
Camera
   |
   | raw frame
   v
Frame Capture
   |
   | frame
   v
Frame Validation
   |
   | valid frame
   v
Preprocessing
   |
   | processed frame
   v
YOLO Detector
   |
   | raw model result
   v
Detection Processing
   |
   | Detection[]
   v
Renderer
   |
   | annotated frame
   v
Display
   |
   v
Keyboard Input
   |
   +------ continue ------> next frame
   |
   +------ exit ----------> shutdown
```

No persistent frame storage exists in this flow.

---

# 30. Error Data Flow

```
Component Failure
       |
       v
Component Boundary
       |
       v
Exception / Failure Result
       |
       v
Central Error Handling
       |
       +---- recoverable ----> Recovery
       |                          |
       |                          +--> success --> RUNNING
```

```
       |                          |
       |                          +--> failure --> Escalation
       |
       +---- unrecoverable --> Shutdown
       |
       v
Logging
       |
       v
Resource Cleanup
       |
       v
STOPPED / FAILED
```

Expected operational failures should be represented through controlled failure results where practical.

Exceptions remain appropriate for unexpected or exceptional conditions.

---

# 31. Error Handling Philosophy

The system distinguishes between:

## Expected operational failures

Examples:

- Camera temporarily fails.
- Frame is invalid.
- Camera disconnects.
- Display becomes unavailable.
- Model file is missing.

These should be handled deliberately.

## Programming defects

Examples:

- Incorrect assumptions inside application code.
- Unexpected attribute errors.
- Broken internal state.
- Logic errors.

These should not be silently swallowed.

The architecture should not use:

```
except Exception:
    pass
```

as a general recovery mechanism.

Unexpected exceptions should be:

1. Captured at an appropriate top-level boundary.
2. Logged with stack trace.
3. Converted into controlled shutdown behavior.
4. Followed by resource cleanup.

---

# 32. Error Severity

Recommended classification:

| Severity | Meaning |
|---|---|
| DEBUG | Detailed diagnostic information |
| INFO | Normal lifecycle/operational event |
| WARNING | Abnormal condition that may recover |
| ERROR | Failure affecting functionality |
| CRITICAL | Severe failure requiring immediate termination or escalation |

Examples:

```
INFO:
Application started.

INFO:
Camera opened successfully.

DEBUG:
Frame dimensions: 640x480.

WARNING:
Camera read failed, consecutive failures: 2.

ERROR:
Camera recovery failed.

CRITICAL:
Application initialization failed.
```

---

# 33. Logging Architecture

Suggested module:

`logger.py`

Python's standard `logging` package is the logging foundation.

The application should have centralized logging initialization rather than creating independent logging systems in every module.

Conceptual architecture:

```
Application
    |
    v
Application Logger
    |
    +---------- Console Handler
    |
    +---------- File Handler
```

Modules obtain module-specific loggers:

```
application
camera
detector
preprocessing
detection
renderer
```

This allows logs to identify their source without requiring each component to implement its own logging infrastructure.

---

# 34. Log Format

A useful format should contain at minimum:

```
timestamp
level
component/module
message
```

Example:

```
2026-10-05 00:35:12 INFO camera - Camera 0 opened successfully
2026-10-05 00:35:14 DEBUG detector - Inference completed in 18.4 ms
2026-10-05 00:35:14 DEBUG detection - 3 valid detections
2026-10-05 00:35:20 WARNING camera - Camera read failure (2 consecutive)
```

Exception logs should include stack traces where appropriate.

---

# 35. Logging Levels

## INFO

Use for significant application lifecycle events:

- Startup
- Configuration summary
- Camera opened

- Model loaded
- Device selected
- Recovery succeeded
- Shutdown initiated
- Shutdown completed

# DEBUG

Use for detailed diagnostics:

- Frame dimensions
- Frame validation
- Detection count
- Detection classes
- Confidence values
- Bounding boxes
- Inference timing
- Processing timing
- State transitions
- Recovery attempts

DEBUG logging should not automatically mean logging every internal operation for every frame indefinitely.

# WARNING

Use for recoverable abnormal conditions:

- Temporary camera failure
- Invalid frame
- Recovery attempt
- Unsupported optional configuration
- Display warning

# ERROR

Use for failed operations:

- Camera recovery failure
- Inference failure

- File I/O failure
- Display failure
- Model operation failure

### CRITICAL

Use for application-level conditions requiring termination:

- Model cannot initialize.
- Required runtime is unavailable.
- Application cannot enter a valid operating state.
- Critical initialization failure.

---

# 36. TRACE-Level Concept

Python's standard `logging` module does not provide a built-in `TRACE` level.

If extremely verbose tracing is eventually required, the application may define a custom numeric logging level below DEBUG.

However, V1 does not require this.

DEBUG plus controlled sampling is sufficient.

A custom TRACE level should not be introduced merely to satisfy terminology.

---

# 37. Log Flood Prevention

The application must use state-aware and rate-limited logging.

Repeated errors must not produce one log entry per processing-loop iteration indefinitely.

For example:

```
Camera read failure begins
        |
        v
WARNING immediately

Failures continue
        |
        v
Suppress repeated messages

Periodic status
        |
        v
WARNING:
Camera still failing; 50 failures over 10 seconds
```

```
Recovery succeeds
        |
        v
INFO:
Camera recovered after 53 failures
```

A reusable repeated-error mechanism should be preferred over separate special-case implementations.

Conceptual mechanism:

```
Failure key
    |
    +-- occurrence count
    +-- first occurrence time
    +-- last logged time
    +-- suppressed count
```

When the condition ends, the logger can report a summary:

```
Camera read failure condition ended.
Occurrences: 53.
Suppressed log messages: 47.
Duration: 10.2 seconds.
```

---

# 38. Logging Failure Handling

Logging must never become a secondary infinite failure loop.

Potential failures:

- Log directory cannot be created.
- File cannot be opened.
- Permission denied.
- Disk full.
- File write failure.
- Handler initialization failure.

Startup behavior:

If file logging cannot be initialized, the application may continue with console logging if the operational environment permits it.

This should be explicitly logged through the available logging path.

The application should not repeatedly attempt to create the same failed log file on every frame.

Runtime handler failures must not result in:

```
logging error
    -> log logging error
        -> log logging error
```

```
        -> ...
```

Logging error handling must not recursively log through the failed handler.

Where possible:

- Keep console logging independent from file logging.
- Disable a failed file handler.
- Emit a single warning through a surviving handler.
- Continue or terminate based on whether file logging is considered mandatory.

For V1, file logging should normally be considered non-critical unless the deployment explicitly requires it.

---

# 39. Log Rotation

Because the application is intended for sustained operation, unbounded log growth should be avoided.

A rotating file handler is appropriate.

V1 configuration should define:

- Maximum log file size.
- Number of retained files.

Exact values are deployment parameters rather than architectural requirements.

Verbose DEBUG logging should not be enabled indefinitely during high-FPS operation without considering disk usage.

---

# 40. Performance Observability

Performance metrics should be measured without substantially affecting the processing loop.

Useful metrics:

```
FPS
Frame processing time
Inference time
Frame capture time where measurable
Detection count
Dropped/skipped frames
Camera read failures
Inference failures
Recovery attempts
```

CPU and memory measurements should be sampled periodically rather than measured through expensive operations on every frame.

GPU monitoring should not be required in the core loop.

---

# 41. Performance Measurement

Recommended runtime measurements:

```
Capture time
Preprocessing time
Inference time
Detection processing time
Rendering/display time
Total frame time
Effective FPS
```

The application should distinguish:

```
Inference FPS
```

from:

```
End-to-end application FPS
```

The latter is the more meaningful V1 user-facing metric.

A rolling average is preferable to reporting highly noisy instantaneous FPS.

---

# 42. Performance Requirements

V1 should target real-time behavior rather than a fixed universal FPS that cannot be guaranteed across all hardware.

Recommended baseline:

- Target: approximately 30 displayed/processed FPS where hardware and selected model permit.
- Hard real-time frame budget for 30 FPS: approximately 33.3 ms per processed frame.
- Preferred processing budget: approximately 25 ms/frame on the defined reference workload.
- Default processing resolution: 640×480 unless model/hardware testing establishes another baseline.
- Startup target: approximately 4 seconds excluding unavoidable model/runtime initialization variation.
- Sustained operation: at least 60 minutes for release qualification.
- Frame processing should not routinely fall below approximately 25 FPS on the reference workload.

These values are acceptance targets for a defined test environment, not universal guarantees.

The reference hardware and model must be documented during performance qualification.

---

# 43. CPU and GPU Utilization

CPU/GPU utilization is environment-dependent.

Rather than enforcing an arbitrary CPU percentage, V1 should establish acceptable resource usage through benchmark testing.

The system should record:

- Average CPU utilization.
- Peak CPU utilization.
- GPU utilization where available.
- Memory usage.
- VRAM usage where available.

Performance testing should determine whether the selected model and resolution satisfy the application's real-time requirement.

---

# 44. Memory Management

V1 should avoid retaining frames unnecessarily.

The main loop should operate conceptually as:

```
capture frame
    ->
process
    ->
render
    ->
display
    ->
discard transient data
    ->
next frame
```

There should be no unbounded frame queue.

The application should not accumulate detection results across frames because V1 does not implement tracking or analytics.

---

# 45. Threading and Concurrency Decision

V1 should use a single-threaded architecture.

The primary loop is:

```
capture
 -> validate
 -> infer
 -> process
 -> render
 -> display/input
 -> repeat
```

This is preferred because:

1. V1 does not require concurrent independent workloads.
2. It eliminates shared-frame synchronization.
3. It eliminates queue/backpressure complexity.
4. Exception propagation is straightforward.
5. Shutdown coordination is simpler.
6. Resource ownership is clear.
7. Debugging is easier.
8. A camera frame does not need to be processed independently of the capture operation in the initial product.

Threading should only be introduced if profiling demonstrates that sequential processing cannot meet the defined performance requirements.

---

# 46. Future Concurrency Extension

If later versions require higher throughput, the architecture can evolve toward:

```
Camera Thread
      |
      v
Bounded Frame Queue
      |
      v
Inference Worker
      |
      v
Detection Queue
      |
      v
Renderer
```

If this occurs, the queue must be bounded.

For real-time detection, dropping stale frames may be preferable to allowing latency to grow indefinitely.

V1 does not implement this architecture.

---

# 47. Detection Output Boundary

The detector is an important abstraction boundary.

Internally:

```
YOLO-specific result
```

Externally:

```
List[Detection]
```

This is intentional.

The application should not contain code throughout the system that directly accesses fields of a specific YOLO result object.

This permits:

```
Ultralytics YOLO
        |
        v
Detector Adapter
        |
        v
Application Detection[]
```

and later:

```
Another Model
        |
        v
Another Detector Adapter
        |
        v
Application Detection[]
```

without changing rendering, configuration, or application-control logic.

---

# 48. Project Structure

A practical V1 structure is:

```
yolo_detection_app/
│
├── main.py
├── config.py
├── camera.py
├── detector.py
├── detection.py
├── preprocessing.py
├── renderer.py
├── logger.py
├── errors.py
│
├── models/
│   └── <pretrained-model-file>
```

```
├── logs/
│
└── tests/
    ├── test_camera.py
    ├── test_detector.py
    ├── test_detection.py
    ├── test_preprocessing.py
    ├── test_renderer.py
    ├── test_logging.py
    └── test_application.py
```

The structure is intentionally small.

---

# 49. Module Responsibilities

## `main.py`

Responsible for:

- Starting the application.
- Constructing/initializing the application.
- Invoking the lifecycle.
- Providing the top-level exception boundary.

Must not contain detailed implementation of camera, YOLO, or rendering behavior.

---

## `config.py`

Responsible for:

- Configuration values.
- Configuration validation.
- Default settings.

Must not contain runtime state.

---

## `camera.py`

Responsible for:

- Camera abstraction.
- Camera lifecycle.
- Frame acquisition.

- Camera recovery.

Must not perform inference or rendering.

---

## `detector.py`

Responsible for:

- Model loading.
- Device initialization.
- Inference.
- Model result conversion.

Must not manage the camera or display.

---

## `detection.py`

Responsible for:

- Detection data model.
- Detection validation.
- Confidence filtering.
- Bounding-box validation.

Must not access the camera or OpenCV windows.

---

## `preprocessing.py`

Responsible for:

- Frame validation.
- Model-required preprocessing.

Must not perform object detection or display.

---

## `renderer.py`

Responsible for:

- Bounding-box drawing.
- Label rendering.
- Display.

- Keyboard input associated with the display.

Must not own the camera or model.

---

## `logger.py`

Responsible for:

- Logging initialization.
- Handler configuration.
- Formatting.
- Rotation.
- Log-level configuration.
- Controlled repeated-error reporting where appropriate.

Must not contain application business logic.

---

## `errors.py`

Responsible for application-specific exception types where they improve clarity.

Potential categories:

```
ApplicationError
ConfigurationError
CameraError
CameraOpenError
CameraReadError
CameraRecoveryError
ModelError
ModelLoadError
InferenceError
RenderingError
```

Not every error requires a custom exception.

Custom exception classes should be introduced where they materially improve error classification or control flow.

---

# 50. Dependency Direction

Preferred dependency direction:

```
main/application
       |
       +---- config
       +---- logger
       +---- camera
```

```
       +---- detector
       +---- preprocessing
       +---- detection
       +---- renderer
       +---- errors
```

Specialized components should not import the application controller.

For example:

`camera.py`

must not call:

`application.shutdown()`

directly.

Instead, it should report failure to its caller.

This prevents circular dependencies and keeps lifecycle ownership centralized.

---

# 51. Application Startup Sequence

Startup should follow a deterministic sequence.

```
1. Process starts
       |
2. Initialize minimal logging
       |
3. Load configuration
       |
4. Validate configuration
       |
5. Initialize full logging configuration
       |
6. Initialize application state
       |
7. Initialize camera
       |
8. Verify camera is operational
       |
9. Initialize YOLO model
       |
10. Verify detector readiness
       |
11. Initialize display
       |
12. Enter RUNNING state
       |
13. Start processing loop
```

If a required initialization step fails:

```
Failure
   |
   v
Log
   |
```

```
   v
Cleanup already-initialized resources
   |
   v
Shutdown
```

The application must support partial initialization cleanup.

---

# 52. Runtime Sequence

For each iteration:

```
1. Check whether shutdown has been requested.
2. Capture frame.
3. Validate frame.
4. Preprocess if required.
5. Run YOLO inference.
6. Convert raw results.
7. Validate detections.
8. Apply confidence threshold.
9. Render detections.
10. Display frame.
11. Process keyboard input.
12. Update performance metrics.
13. Repeat.
```

The order may be optimized after profiling, but responsibilities should remain separated.

---

# 53. No-Detection Behavior

No detections is normal.

Example:

```
Frame
  |
YOLO
  |
0 detections
  |
Renderer
  |
Display original frame
```

The application continues normally.

It should not:

- Raise an exception.
- Shut down.
- Log an error every frame.
- Attempt recovery.

At DEBUG level, detection count may be reported using sampling/rate limiting.

---

# 54. Detection Processing Data Flow

```
YOLO Result
    |
    v
Extract candidate detections
    |
    v
Validate class ID
    |
    v
Resolve class name
    |
    v
Validate confidence
    |
    v
Apply threshold
    |
    v
Validate bounding box
    |
    +---- invalid --> discard + diagnostic
    |
    v
Detection[]
```

An invalid individual detection should not necessarily invalidate other valid detections from the same frame.

---

# 55. Failure Recovery Policy

V1 uses bounded recovery.

The basic policy is:

```
Failure
  |
  v
Classify
  |
  +---- transient --> bounded retry
  |
  +---- recoverable --> recovery procedure
  |
  +---- unrecoverable --> shutdown
```

Recovery must have:

- Maximum attempts.
- Retry delay.

- Failure escalation.
- Logging policy.

No subsystem may retry indefinitely.

---

# 56. Retry Policy

Retries should be limited to operations where retrying is meaningful.

Good candidates:

- Camera reads.
- Camera reconnection.
- Potentially transient runtime operations.

Poor candidates:

- Loading a missing model file.
- Loading a corrupt model repeatedly.
- Writing repeatedly to a known-invalid path.
- Reinitializing a permanently unsupported runtime configuration.

A reasonable V1 retry strategy is:

```
attempt
wait
attempt
wait
attempt
failure escalation
```

A bounded exponential backoff may be used for recovery, but a simple fixed delay is sufficient for V1.

For example:

```
0.25 s
0.5 s
1 s
2 s
maximum delay
```

The exact values are configuration/test parameters.

---

# 57. Repeated Failure State

The architecture should track repeated failures using counters and timestamps.

For example:

```
FailureTracker
    failure_type
    consecutive_count
    total_count
    first_failure_time
    last_failure_time
    last_logged_time
    suppressed_count
```

This mechanism can conceptually serve:

- Camera errors
- Inference errors
- Display errors
- File errors

It should not be unnecessarily coupled to a particular subsystem.

---

# 58. Error Escalation

Example:

```
Camera read failure #1
    WARNING

Camera read failure #2
    suppressed/rate-limited

Camera recovery attempt #1
    INFO/DEBUG

Recovery failure
    WARNING

Maximum recovery attempts reached
    ERROR

Application shutdown
    INFO
```

The final shutdown log should summarize the failure where useful.

---

# 59. Resource Ownership

Each component owns the resources it creates.

Examples:

```
Camera component
    owns camera capture object

Renderer
```

```
    owns display/window resources

Logger
    owns logging handlers

Detector
    owns model/runtime references it explicitly creates
```

The application controller owns lifecycle ordering but should not manipulate internal resource objects directly.

---

# 60. Cleanup Ordering

Recommended shutdown order:

```
1. Stop accepting new work.
2. Stop frame-processing loop.
3. Release camera.
4. Destroy OpenCV windows.
5. Release model/runtime resources where required.
6. Record cleanup result.
7. Flush/close log handlers.
8. Transition to STOPPED.
```

The exact model-release operation depends on the selected runtime.

Cleanup should be defensive because some resources may never have been initialized.

---

# 61. Idempotent Cleanup

Shutdown should be safe when:

- Camera was never opened.
- Model failed during initialization.
- Display was never initialized.
- Logging is partially initialized.
- Cleanup is called more than once.

Conceptually:

```
if camera exists:
    release

if windows initialized:
    destroy

if model resources require release:
    release

if logging initialized:
    close handlers
```

A failed cleanup operation should not prevent other independent resources from being released.

---

# 62. Unexpected Exception Handling

The top-level application boundary should catch unexpected exceptions.

Conceptually:

```
try:
    startup
    run
except KeyboardInterrupt:
    controlled shutdown
except expected application failure:
    log and shutdown
except Exception:
    log stack trace
    mark FAILED
finally:
    cleanup
```

The broad `Exception` handler exists at the application boundary as a final safety mechanism.

It should not replace specific exception handling inside individual components.

Programming defects should remain visible through stack traces.

---

# 63. KeyboardInterrupt

`KeyboardInterrupt` is treated as a controlled shutdown request.

Behavior:

```
KeyboardInterrupt
      |
      v
INFO: shutdown requested
      |
      v
cleanup
      |
      v
STOPPED
```

A stack trace is generally unnecessary for an intentional keyboard interruption.

---

# 64. Out-of-Memory Conditions

Out-of-memory conditions are difficult to reliably recover from within the same process.

The architecture should therefore treat severe memory exhaustion as effectively unrecoverable.

The system should avoid creating conditions that make OOM likely by:

- Avoiding unbounded queues.
- Avoiding frame accumulation.
- Avoiding unnecessary copies.
- Avoiding persistent detection histories.
- Avoiding excessive logging.

If the runtime raises a catchable memory exception, the application may log it and attempt controlled cleanup, but it should not attempt repeated model or frame allocation indefinitely.

---

# 65. File I/O Failures

Potential V1 file operations include:

- Model file access.
- Log file access.
- Directory creation.

A model file failure is startup-critical.

A log file failure is normally non-critical if console logging remains functional.

The application should distinguish:

```
Required file
```

from:

```
Optional diagnostic file
```

rather than applying one policy to every I/O failure.

---

# 66. Configuration Failure

Configuration validation occurs before normal execution.

Examples:

```
camera_index < 0
invalid confidence threshold
invalid retry count
empty model path
unsupported device value
invalid logging level
invalid resolution
```

Behavior:

```
Invalid configuration
       |
       v
ConfigurationError
       |
       v
Log error
       |
       v
Do not initialize runtime resources unnecessarily
       |
       v
Exit
```

Configuration errors should not trigger repeated runtime retries.

---

# 67. Security and Privacy

V1 is local-only.

The application:

- Does not transmit camera frames.
- Does not upload frames.
- Does not use cloud inference.
- Does not require networking.
- Does not expose a web API.
- Does not require a database.
- Does not persist captured frames.
- Does not implement user accounts.

Camera data remains within the local process unless the operating system or external software independently accesses the camera.

---

# 68. Logging Privacy

Logs should not contain:

- Camera image contents.
- Raw image data.
- Base64-encoded frames.
- Personal identity information.
- Unnecessary filesystem secrets.

- Authentication credentials.
- Environment secrets.
- Full sensitive environment variables.

Logging:

```
class
confidence
bounding box
FPS
timing
error type
```

is acceptable for debugging because these are application diagnostics.

If model paths or filesystem paths could contain sensitive information, the logging policy should avoid unnecessarily exposing them.

---

# 69. Data Persistence

V1 uses transient runtime data only.

```
Camera frame
    |
    v
Inference
    |
    v
Detection
    |
    v
Rendering
    |
    v
Display
    |
    v
Discard
```

No detection history is stored.

No frame archive is created.

No database is used.

---

# 70. Non-Functional Requirements

## Performance

NFR-PERF-001:

The reference configuration should target approximately 30 processed/displayed FPS where supported by the selected model and hardware.

NFR-PERF-002:

A 30 FPS workload has an end-to-end processing budget of approximately 33.3 ms/frame.

NFR-PERF-003:

A preferred target is approximately 25 ms/frame on the defined reference workload.

NFR-PERF-004:

Performance instrumentation must distinguish inference latency from total frame-processing latency.

---

# Reliability

NFR-REL-001:

Transient camera failures shall not immediately terminate the application.

NFR-REL-002:

All retry/recovery loops shall be bounded.

NFR-REL-003:

An unrecoverable required-component failure shall cause controlled shutdown.

NFR-REL-004:

Resource cleanup shall execute after both normal and abnormal termination.

---

# Maintainability

NFR-MAIN-001:

Camera, detector, rendering, configuration, logging, and lifecycle responsibilities shall remain separated.

NFR-MAIN-002:

YOLO-specific result structures shall not propagate outside the detector boundary.

NFR-MAIN-003:

Operational parameters shall not be hard-coded throughout the application.

---

# Extensibility

NFR-EXT-001:

A new camera implementation should be replaceable without changing detection-processing logic.

NFR-EXT-002:

A different detector implementation should be replaceable without changing rendering logic.

NFR-EXT-003:

Future tracking/counting functionality should be insertable after detection processing without redesigning camera management.

---

# Observability

NFR-OBS-001:

Startup and shutdown must be logged.

NFR-OBS-002:

Component failures must include component context.

NFR-OBS-003:

Unexpected exceptions must include stack traces.

NFR-OBS-004:

Repeated failures must be rate-limited or aggregated.

---

# Portability

NFR-PORT-001:

Core application logic should avoid OS-specific APIs.

NFR-PORT-002:

The application should target supported Python/OpenCV/YOLO environments on Windows and Linux, with macOS support dependent on the selected YOLO/PyTorch runtime and available hardware acceleration.

---

# Testability

NFR-TEST-001:

Camera and detector implementations must be replaceable by test doubles.

NFR-TEST-002:

Detection validation must be testable independently from hardware.

NFR-TEST-003:

Application shutdown must be testable without requiring a physical camera.

---

# 71. Testing Architecture

Testing should occur at multiple levels.

```
Unit Tests
    |
    v
Component/Integration Tests
    |
    v
End-to-End Tests
    |
    v
Performance Tests
    |
    v
Stress/Fault Testing
```

---

# 72. Unit Testing

Unit tests should cover deterministic logic independently of hardware.

### Configuration

Test:

- Valid configuration.
- Invalid camera index.
- Invalid threshold.
- Invalid device.
- Invalid retry values.
- Invalid model path.

### Frame validation

Test:

- `None`.
- Empty frame.
- Wrong shape.
- Wrong data type.
- Valid frame.

### Detection validation

Test:

- Valid detection.
- Confidence below threshold.
- Confidence at threshold.
- Confidence above threshold.
- Invalid coordinates.
- Reversed coordinates.
- Negative coordinates.
- Coordinates beyond frame dimensions.
- Zero-area boxes.
- Multiple detections.
- Empty detections.

### Performance calculations

Test:

- Frame-time calculations.
- FPS calculation.
- Zero/near-zero elapsed time.
- Counter resets.

---

# 73. Camera Unit Testing

The real camera should not be required for normal unit tests.

A fake/mock camera should simulate:

```
open succeeds
open fails
read succeeds
read fails once
read fails repeatedly
disconnects
reconnects
reconnect fails
release succeeds
release raises
```

The tests should verify both application behavior and retry limits.

---

# 74. Detector Unit Testing

A fake detector should simulate:

```
model loads
model fails to load
inference succeeds
no detections
multiple detections
malformed result
inference exception
repeated inference failure
```

This verifies the application independently of an actual GPU/model runtime.

---

# 75. Rendering Tests

Rendering can be tested with:

- Synthetic frames.
- Known detection objects.
- Mocked OpenCV display functions.

Verify:

- Bounding boxes are drawn.
- Labels are generated correctly.
- Multiple detections render.
- Empty detection lists render without failure.
- Exit input is recognized.
- Display failures are handled.

---

# 76. Logging Tests

Logging tests should verify:

- Logging initializes.
- Console handler works.
- File handler works.
- Log levels are respected.
- Exceptions include stack traces.
- Rotation configuration is respected.

- Repeated failures are rate-limited.
- Suppressed errors are summarized.
- Logging handler failure does not cause recursion.
- Shutdown closes handlers.

---

# 77. Integration Testing

Integration tests should verify boundaries between:

```
Camera -> preprocessing
preprocessing -> detector
detector -> detection processor
detection processor -> renderer
renderer -> application controller
```

The tests should use mocks/fakes where external hardware or runtime dependencies would make tests unreliable.

---

# 78. End-to-End Testing

A controlled test environment should verify:

```
Start application
    |
Camera opens
    |
Model loads
    |
Live feed appears
    |
Objects detected
    |
Boxes and labels displayed
    |
Exit key pressed
    |
Shutdown
    |
Camera released
    |
Window destroyed
    |
Logs closed
```

The test should verify that the process terminates cleanly.

---

# 79. Fault-Injection Testing

Required fault scenarios include:

### Camera

- Camera unavailable.
- Camera busy.
- Camera initialization exception.
- Single read failure.
- Repeated read failures.
- Camera disconnect.
- Camera reconnect success.
- Camera reconnect failure.

### Frames

- `None`.
- Empty frame.
- Invalid dimensions.
- Invalid dtype.
- Continuous invalid frames.

### Model

- Missing model.
- Corrupt model.
- Invalid model path.
- Runtime incompatibility.
- GPU initialization failure.
- Single inference exception.
- Repeated inference exceptions.

### Detection

- No detections.
- Multiple detections.
- Invalid class ID.
- Invalid confidence.

- Invalid coordinates.
- Out-of-bounds coordinates.

### Display

- Window initialization failure.
- Display exception.
- Window closed.
- Keyboard input failure.

### Logging

- Directory creation failure.
- File permission failure.
- Write failure.
- Disk-full simulation.
- Handler failure.

### Lifecycle

- KeyboardInterrupt.
- Unexpected exception.
- Failure halfway through initialization.
- Repeated shutdown calls.
- Cleanup exception.

---

# 80. Performance Testing

Performance testing should measure:

```
Average FPS
Minimum FPS
Frame-time average
Frame-time 95th percentile
Frame-time maximum
Inference latency
Camera capture latency where measurable
CPU utilization
GPU utilization where applicable
Memory usage
VRAM usage where applicable
Dropped/invalid frames
Detection count
Startup time
Shutdown time
```

Testing must specify:

- Hardware.
- OS.
- Python version.
- Model.
- Model size.
- Camera resolution.
- Confidence threshold.
- Device.
- Test duration.

Without this information, FPS figures are not reproducible.

---

# 81. Stress Testing

The application should undergo sustained operation.

Recommended initial qualification:

```
15 minutes: basic stability
60 minutes: release qualification
Longer duration: deployment-specific validation
```

Stress testing should look for:

- Memory growth.
- Increasing latency.
- FPS degradation.
- Log-file growth.
- Camera instability.
- GPU memory growth.
- Resource leaks.
- Window/display degradation.

---

# 82. Regression Testing

Every defect discovered in development should result in a regression test where practical.

Examples:

```
Camera recovery bug
    -> camera recovery regression test

Log flooding bug
    -> repeated-error logging test

Cleanup failure
    -> partial-initialization cleanup test

Invalid bounding-box bug
    -> detection validation test
```

This prevents previously fixed lifecycle and reliability problems from returning.

---

# 83. Mocking Strategy

External dependencies should be mocked or replaced with test doubles.

Primary candidates:

```
Camera
YOLO model
OpenCV display
Keyboard input
File system
Logging handlers
Time source
```

Time should be injectable or abstracted where precise retry/rate-limit tests require deterministic timing.

---

# 84. Testable Interfaces

The architecture improves testability by keeping interfaces narrow.

Example:

```
Camera
    read()
    reconnect()
    release()

Detector
    detect(frame)

Renderer
    render(frame, detections)
    show(frame)
    exit_requested()

Logger
    initialize()
```

The exact API is implementation-defined, but the principle is that each component should expose only the behavior required by its consumers.

---

# 85. Architecture Decision Records

## ADR-001: Use YOLO

Decision:

Use a pretrained YOLO model for V1 object detection.

Reason:

YOLO provides a mature single-stage object-detection approach suitable for real-time inference.

Consequence:

The system depends on a deep-learning runtime and model weights.

---

## ADR-002: Use Ultralytics as the Initial YOLO Adapter

Decision:

Use Ultralytics YOLO as the preferred initial model interface.

Reason:

It provides a practical Python API and pretrained model support.

Consequence:

The detector module must isolate Ultralytics-specific result handling.

---

## ADR-003: Use OpenCV for Camera and Display

Decision:

Use OpenCV for local camera acquisition and display.

Reason:

It provides the required camera, image, window, keyboard, and drawing primitives without introducing another GUI framework.

Consequence:

Display behavior is tied to an OpenCV-compatible local graphical environment.

---

## ADR-004: Abstract Camera Access

Decision:

Camera operations are encapsulated behind a Camera component.

Reason:

Camera implementation details should not leak into application logic.

Consequence:

Future cameras can be added or replaced without redesigning detection logic.

---

## ADR-005: Centralize Logging

Decision:

Use Python's standard logging framework through a centralized logging module.

Reason:

Logging is cross-cutting infrastructure and must behave consistently across components.

Consequence:

Modules use component-specific loggers rather than independent logging systems.

---

## ADR-006: Bound Recovery

Decision:

All recovery loops are bounded.

Reason:

Infinite retries can prevent shutdown, consume CPU, and flood logs.

Consequence:

Persistent failures eventually escalate to controlled shutdown.

---

## ADR-007: Single-Threaded V1

Decision:

Use a single processing loop.

Reason:

V1 does not require concurrency and a sequential pipeline is easier to reason about, test, and shut down.

Consequence:

The achievable FPS depends on the combined capture, inference, rendering, and display latency.

---

## ADR-008: No Frame Persistence

Decision:

Do not save captured frames.

Reason:

Frame storage is outside V1 requirements and creates additional privacy, storage, and lifecycle concerns.

Consequence:

Recording can be added later at a defined pipeline boundary.

---

## ADR-009: No Database

Decision:

Do not use a database.

Reason:

V1 does not require persistent detection history, analytics, or application state.

Consequence:

Future analytics can introduce persistence without contaminating the V1 detection pipeline.

---

## ADR-010: No Networking

Decision:

No networking is included in V1.

Reason:

The product objective is local real-time inference.

Consequence:

Remote cameras, cloud inference, APIs, dashboards, and distributed processing remain future capabilities.

---

# 86. Failure Mode Table

| Failure | Detection | Severity | Recovery | Retry Limit | Logging | Final Behavior |
|---|---|---|---|---|---|---|
| Camera unavailable | Camera open fails | ERROR | Optional bounded retry | Configured | Immediate | Shutdown |
| Camera busy | Camera cannot open | ERROR | Retry only if configured | Bounded | Immediate | Shutdown |
| Camera initialization exception | Exception | ERROR | Usually none | 0 or configured | Stack trace | Shutdown |
| Single camera read failure | `read()` failure | WARNING | Retry | Bounded | Immediate/ rate-limited | Continue if recovered |
| Repeated camera reads fail | Consecutive failures | ERROR | Reconnect | Bounded | Aggregated | Shutdown if recovery fails |
| Camera disconnect | Read failure/state loss | ERROR | Reconnect | Bounded | State-aware | Shutdown if unrecoverable |
| Camera reconnect succeeds | Valid camera restored | INFO | Resume | N/A | Summary | Continue |
| Invalid frame | Validation failure | WARNING | Skip frame | N/A | Rate-limited | Continue |
| Continuous invalid frames | Persistent invalid input | ERROR | Camera recovery | Bounded | Aggregated | Shutdown if persistent |
| Model missing | File check/load failure | CRITICAL | None | 0 | Immediate | Shutdown |
| Model corrupt | Model load exception | CRITICAL | None | 0 | Stack trace | Shutdown |
| Runtime incompatible | Model/ runtime exception | CRITICAL | None | 0 | Stack trace | Shutdown |
| GPU unavailable with explicit GPU config | Runtime/ device check | ERROR | None unless configured | 0 | Immediate | Shutdown |
| GPU failure during inference | Runtime exception | ERROR | Optional controlled recovery | Bounded | Stack trace | Shutdown if unusable |
| Single inference failure | Detector exception | ERROR | Skip frame if detector remains valid | Configured | Rate-limited | Continue if safe |
| Repeated inference failures | Failure counter | CRITICAL | Detector recovery if supported | Bounded | Aggregated | Shutdown |
| No detections | Empty result | Normal | None | N/A | None/ DEBUG sampled | Continue |
| Invalid detection | Validation | WARNING | Discard | N/A | Rate-limited | Continue |

| Failure | Detection | Severity | Recovery | Retry Limit | Logging | Final Behavior |
|---|---|---|---|---|---|---|
| | failure | | detection | | | |
| Display failure | OpenCV exception/failure | ERROR | Generally none | 0/ configured | Immediate | Shutdown |
| Window closed | Window-state check | INFO | User shutdown | 0 | Immediate | Shutdown |
| KeyboardInterrupt | Python interrupt | INFO | Controlled shutdown | 0 | Immediate | Shutdown |
| Configuration invalid | Validation failure | CRITICAL | Correct configuration | 0 | Immediate | Shutdown |
| Model file I/O failure | File exception | CRITICAL | None | 0 | Stack trace | Shutdown |
| Log directory unavailable | Directory error | WARNING/ ERROR | Console fallback | Bounded | Console | Continue if possible |
| Log file write failure | Handler error | WARNING | Disable failed handler | 0/recovery limited | Console | Continue |
| Disk full | File write failure | WARNING/ ERROR | Disable/ rotate handler | Bounded | Console | Continue if possible |
| Unexpected programming exception | Top-level exception | CRITICAL | None | 0 | Full traceback | Shutdown |
| Partial initialization failure | Startup exception | ERROR/ CRITICAL | Cleanup initialized resources | 0 | Stack trace | Shutdown |
| Cleanup exception | Exception during release | ERROR | Continue other cleanup | 0 | Immediate | Attempt full cleanup |
| OOM/resource exhaustion | Runtime/ system error | CRITICAL | Controlled cleanup | 0 | Best effort | Shutdown |

---

# 87. Infinite Failure-Loop Prevention

The architecture explicitly prohibits unbounded retry behavior.

Every retry operation must have at least one of:

```
Maximum attempts
Maximum elapsed time
Explicit state transition
Failure escalation
```

Examples:

```
Camera reconnect:
    bounded attempts

Model loading:
    no repeated loading for known-invalid configuration

Logging:
```

```
    failed handler disabled rather than repeatedly retried

Display:
    failure escalates rather than repeatedly reopening windows

Shutdown:
    cleanup attempted once per resource, with defensive state checks
```

The processing loop itself is naturally continuous during normal operation, but every failure path inside that loop must eventually return to:

```
continue normally
```

or:

```
shutdown
```

There must be no third state where the application remains indefinitely stuck attempting recovery.

---

# 88. Startup Failure Matrix

```
Logging initialization fails
        |
        +--> console logging available --> continue

Configuration invalid
        |
        +--> shutdown

Camera initialization fails
        |
        +--> shutdown

Model initialization fails
        |
        +--> release camera
        +--> shutdown

Display initialization fails
        |
        +--> release camera
        +--> release model resources
        +--> shutdown
```

This guarantees that resources initialized before a later failure are still released.

---

# 89. Runtime Failure Matrix

```
RUNNING
   |
   +--> Invalid frame
   |       |
   |       +--> skip frame
   |
```

```
   +--> Camera read failure
   |       |
   |       +--> bounded recovery
   |
   +--> Inference failure
   |       |
   |       +--> classify
   |             |
   |             +--> recover/continue
   |             +--> shutdown
   |
   +--> Display failure
   |       |
   |       +--> shutdown
   |
   +--> Exit requested
           |
           +--> shutdown
```

---

# 90. Shutdown Sequence

Normal shutdown:

```
User presses Q
       |
       v
Application controller receives exit request
       |
       v
Transition to SHUTTING_DOWN
       |
       v
Stop main processing loop
       |
       v
Release camera
       |
       v
Destroy OpenCV windows
       |
       v
Release model/runtime resources where applicable
       |
       v
Write cleanup status
       |
       v
Close logging handlers
       |
       v
Transition to STOPPED
```

Unexpected shutdown:

```
Unexpected exception
       |
       v
Top-level exception handler
       |
```

```
       v
Log traceback
       |
       v
Transition to SHUTTING_DOWN
       |
       v
Best-effort cleanup
       |
       v
Close logging
       |
       v
STOPPED / FAILED
```

---

# 91. Cleanup Guarantees

The application should guarantee, as far as the operating environment permits:

- Camera capture is released.
- OpenCV windows are destroyed.
- File handles are closed.
- Log handlers are closed.
- No application-owned background worker remains active.
- No unbounded recovery operation remains active.

The system should not rely solely on process termination to clean up resources.

---

# 92. V1 Scope

## In Scope

V1 implements:

- Local camera selection.
- Camera initialization.
- Live camera feed.
- Continuous frame capture.
- Frame validation.
- Required preprocessing.
- Pretrained YOLO model loading.
- CPU/GPU device selection as supported.

- Per-frame inference.
- Object class identification.
- Confidence extraction.
- Bounding-box extraction.
- Detection validation.
- Confidence filtering.
- Bounding-box rendering.
- Class/confidence display.
- Live display.
- Keyboard-based shutdown.
- Camera failure handling.
- Camera reconnection with bounded recovery where configured.
- Model failure handling.
- Display failure handling.
- Centralized logging.
- Console logging.
- File logging.
- Log rotation where configured.
- Log flooding prevention.
- Performance measurement.
- Clean resource management.
- Unit/integration/E2E testing architecture.

---

# 93. Explicitly Out of Scope

V1 does not implement:

- Object tracking.
- Persistent object IDs.
- Object counting.
- Custom model training.
- Dataset creation.
- Model fine-tuning.

- Model training pipelines.
- Image recording.
- Video recording.
- Frame archiving.
- Cloud processing.
- Remote inference.
- Remote cameras.
- Networking.
- Web APIs.
- Databases.
- Notifications.
- Authentication.
- User accounts.
- Web dashboards.
- Mobile applications.
- Advanced analytics.
- Facial recognition.
- Face identification.
- Persistent detection history.
- Multi-camera processing.

These are future extension points rather than hidden V1 functionality.

---

# 94. Future Extension Architecture

The V1 architecture intentionally creates boundaries for future capabilities.

## Multiple Cameras

Future:

```
CameraManager
    |
    +--> Camera 1
    +--> Camera 2
    +--> Camera N
```

V1 has:

```
Application
    |
    v
Camera
```

The camera abstraction allows this evolution.

---

# Object Tracking

Future insertion point:

```
YOLO Detection
      |
      v
Detection Processing
      |
      v
Tracker
      |
      v
Renderer
```

V1 stops before tracking.

---

# Object Counting

Future:

```
Detection
    |
    v
Tracker / Counter
    |
    v
Analytics
```

No counting state exists in V1.

---

# Recording

Future:

```
Camera
   |
   v
Frame
   +----> Detector
   |
   +----> Recorder
```

The recorder can be added without making the detector responsible for storage.

---

## Alternative Models

Future:

```
Detector Interface
       |
       +--> YOLO
       +--> Other detector
       +--> Hardware accelerator
```

The application consumes the common detection representation.

---

## Custom Models

Future configuration may specify:

```
model path
model metadata
class configuration
device
```

The V1 detector boundary already permits custom model loading without requiring changes to rendering or camera logic.

---

## GUI

Future:

```
Application
    |
    +--> CLI/OpenCV renderer
    |
    +--> Desktop GUI renderer
```

The rendering boundary isolates UI behavior from detection.

---

## Performance Monitoring

Future observability could introduce:

```
Performance Monitor
       |
       +--> FPS
       +--> latency
       +--> CPU
       +--> GPU
       +--> memory
```

V1 retains only lightweight runtime metrics.

---

# 95. Component Boundary Summary

| Component | Primary Responsibility |
|---|---|
| Application | Lifecycle and orchestration |
| Configuration | Operational configuration |
| Camera | Camera lifecycle and frame capture |
| Preprocessing | Frame validation/preprocessing |
| Detector | Model loading and inference |
| Detection | Result validation/filtering |
| Renderer | Annotation and display |
| Logger | Observability and diagnostics |
| Errors | Error classification |

---

# 96. Primary Interfaces

Conceptual interfaces:

```
Camera
    open()
    is_open()
    read()
    reconnect()
    release()

Detector
    load()
    is_ready()
    detect(frame)
    release()

Detection Processor
    validate(results)
    filter(results)

Renderer
    render(frame, detections)
    show(frame)
    exit_requested()
    close()

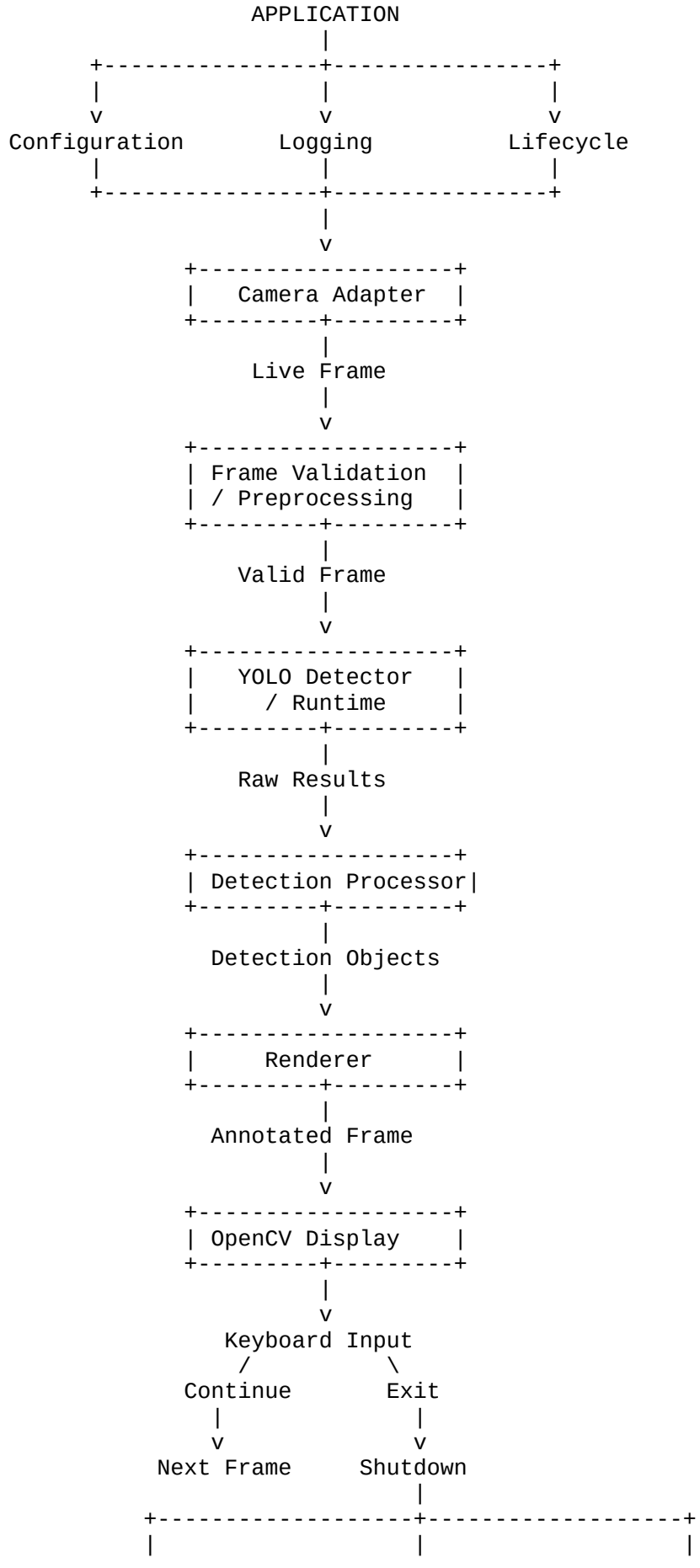
Configuration
    load()
    validate()

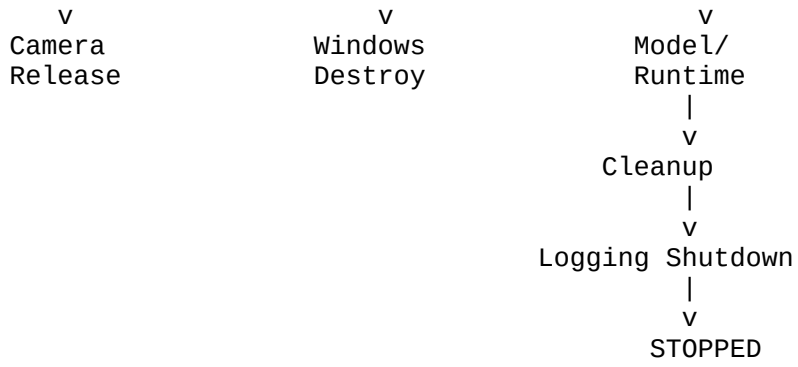
Logging
    initialize()
    get_logger()
    shutdown()
```

These are architectural contracts, not mandatory implementation signatures.

---

# 97. Final End-to-End Architecture






---

# 98. Startup Sequence Summary

```
START
  |
  v
Initialize logging
  |
  v
Load + validate configuration
  |
  v
Initialize camera
  |
  +---- failure --> cleanup --> FAILED
  |
  v
Initialize YOLO
  |
  +---- failure --> camera cleanup --> FAILED
  |
  v
Initialize display
  |
  +---- failure --> cleanup --> FAILED
  |
  v
RUNNING
```

---

# 99. Runtime Sequence Summary

```
RUNNING
   |
   v
Capture frame
   |
   +---- failure --> bounded recovery
   |
   v
Validate frame
   |
   +---- invalid --> skip
   |
```

```
   v
Preprocess
   |
   v
YOLO inference
   |
   +---- failure --> classify/recover/escalate
   |
   v
Validate detections
   |
   v
Apply confidence threshold
   |
   v
Render boxes + labels
   |
   v
Display
   |
   +---- display failure --> shutdown
   |
   v
Read exit key
   |
   +---- exit --> shutdown
   |
   v
Measure/update performance
   |
   v
NEXT FRAME
```

---

# 100. Shutdown Sequence Summary

```
Shutdown Request / Fatal Failure
             |
             v
      SHUTTING_DOWN
             |
             v
       Stop processing
             |
             v
       Release camera
             |
             v
      Destroy OpenCV UI
             |
             v
    Release model/runtime
             |
             v
    Record cleanup status
             |
             v
      Close log handlers
             |
             v
          STOPPED
```

# 101. Key Architectural Principles

The final V1 architecture follows these principles:

1. The application controller owns lifecycle orchestration.
2. The camera is accessed exclusively through a camera abstraction.
3. The YOLO implementation is isolated behind a detector boundary.
4. Application-level detection objects are independent of YOLO-specific result structures.
5. Rendering is isolated from detection and camera management.
6. Configuration is centralized.
7. Logging is centralized and fault tolerant.
8. Operational failures are distinguished from programming defects.
9. Recovery is bounded.
10. Repeated errors are rate-limited or aggregated.
11. No component is permitted to create an uncontrolled retry loop.
12. No frame is persisted in V1.
13. No networking or cloud infrastructure is required.
14. V1 uses a simple single-threaded pipeline.
15. Resource cleanup occurs regardless of the exit path.
16. Cleanup is designed to tolerate partial initialization.
17. Performance instrumentation must not become a significant part of the frame-processing cost.
18. Hardware/model-specific performance claims must be validated against a defined benchmark environment.
19. Future functionality is represented through clean extension boundaries rather than premature implementation.
20. V1 remains a complete usable application rather than merely a framework for a future product.

---

# 102. Architecture Acceptance Criteria

The SAD is considered satisfied by an implementation when the resulting application can demonstrate:

## Functional

- Camera opens successfully.
- Live frames are continuously captured.
- Valid frames reach YOLO.
- Objects are detected.
- Classes are identified.
- Confidence values are available.
- Bounding boxes are available.
- Bounding boxes and labels are displayed.
- Multiple objects are handled.
- No-detection frames continue normally.
- User can terminate the application.

## Reliability

- Camera-open failure does not enter an infinite retry loop.
- Camera read failures are bounded and recoverable where possible.
- Camera disconnects do not create uncontrolled loops.
- Model initialization failure terminates cleanly.
- Inference exceptions do not automatically cause uncontrolled process failure.
- Invalid frames are skipped safely.
- Display failures are handled.
- Unexpected exceptions result in controlled cleanup.

## Observability

- Startup is logged.
- Shutdown is logged.
- Camera lifecycle is logged.
- Model lifecycle is logged.
- Major failures include useful diagnostics.
- Unexpected exceptions include stack traces.
- Repeated failures do not flood logs.
- Performance metrics can be inspected.

### Resource Management

- Camera is released.
- OpenCV windows are destroyed.
- Model/runtime resources are released where applicable.
- File handles are closed.
- Logging handlers are closed.
- Partial initialization does not leave resources active.

### Maintainability

- Camera logic is isolated.
- Detector logic is isolated.
- Rendering logic is isolated.
- Configuration is centralized.
- Logging is centralized.
- Detection representation is application-owned.
- V1 does not depend throughout the codebase on vendor-specific YOLO result structures.

### Testability

- Camera can be mocked.
- Detector can be mocked.
- Display can be mocked.
- Time can be controlled where required.
- Fault conditions can be injected.
- Core detection validation can be tested without hardware.

---

# 103. Final Architectural Position

V1 should be implemented as a local, modular, single-process Python application using OpenCV for camera/display operations and an isolated YOLO detector backed by the required deep-learning runtime.

The core runtime is intentionally simple:

```
Camera
  -> Frame Validation
  -> YOLO Inference
  -> Detection Processing
  -> Rendering
  -> Display
```

```
-> Repeat
```

The complexity belongs primarily in the boundaries around this pipeline: lifecycle management, failure classification, bounded recovery, logging, configuration, resource cleanup, and testing.

This is preferable to introducing concurrency, distributed components, databases, networking, or other infrastructure that V1 does not require.

The resulting architecture is sufficient for an end-to-end V1 product while establishing clean replacement points for a future camera manager, alternative detection models, tracking, counting, recording, GUI layers, analytics, performance monitoring, and other capabilities.

V1 therefore provides a complete object-detection application without coupling future product development to the implementation details of the initial camera, model, rendering, or runtime infrastructure.